Reverse an Array

Difficulty: **Easy** Accuracy: **55.32%** Submissions: **276K+** Points: **2** Average Time: **5m**

You are given an array of integers **arr[]**. You have to **reverse** the given array.

Note: Modify the array in place.

Examples:

Input: arr = [1, 4, 3, 2, 6, 5]
Output: [5, 6, 2, 3, 4, 1]
Explanation: The elements of the array are [1, 4, 3, 2, 6, 5]. After reversing the array, the first element goes to the last position, the second element goes to the second last position and so on. Hence, the answer is [5, 6, 2, 3, 4, 1].

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1115 / 1115

Attempts : Correct / Total
5 / 6
Accuracy : 83%

Time Taken

Java (21) Your Time: 0m 12s

```java
class Solution {
    public void reverseArray(int arr[]) {
        int n = arr.length;
        int j = n-1;
        for(int i = 0; i<j; i++, j--)
        {
            int t = arr[i];
            arr[i] = arr[j];
            arr[j] = t;
        }

    }
}
```

Custom Input Compile & Run Submit

Min and Max in Array

Difficulty: **Basic** Accuracy: **68.55%** Submissions: **408K+** Points: **1** Average Time: **10m**

Given an array **arr[]**. Your task is to find the **minimum** and **maximum** elements in the array.

Examples:

Input: arr[] = [1, 4, 3, 5, 8, 6]
Output: [1, 8]
Explanation: minimum and maximum elements of array are 1 and 8.

Input: arr[] = [12, 3, 15, 7, 9]
Output: [3, 15]
Explanation: minimum and maximum element of array are 3 and 15

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1111 / 1111

Attempts : Correct / Total
2 / 2
Accuracy : 100%

Time Taken

Java (21) Start Timer

```java
class Solution {
    public ArrayList<Integer> getMinMax(int[] arr) {
        ArrayList<Integer> result = new ArrayList<>();

        int min = arr[0];
        int max = arr[0];

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < min) {
                min = arr[i];
            }
            if (arr[i] > max) {
                max = arr[i];
            }
        }

        result.add(min);
        result.add(max);

        return result;

    }
}
```

Custom Input Compile & Run Submit

Kth Smallest

Difficulty: **Medium** Accuracy: **35.17%** Submissions: **742K+** Points: **4** Average Time: **25m**

Given an integer array **arr[]** and an integer **k**, your task is to find and return the **kth smallest** element in the given array.

Note: The kth smallest element is determined based on the sorted order of the array.

Examples :

Input: arr[] = [10, 5, 4, 3, 48, 6, 2, 33, 53, 10], k = 4
Output: 5
Explanation: 4th smallest element in the given array is 5.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1121 / 1121

Attempts : Correct / Total
1 / 1
Accuracy : 100%

Points Scored

Time Taken

Java (21) Start Timer

```java
import java.util.Arrays;
class Solution {
    public int kthSmallest(int[] arr, int k) {
        Arrays.sort(arr);
        return arr[k - 1];
    }
}
```

Custom Input Compile & Run Submit

Union of Arrays with Duplicates

Difficulty: **Easy** Accuracy: **42.22%** Submissions: **494K+** Points: **2** Average Time: **10m**

You are given two arrays **a[]** and **b[]**, return the **Union** of both the arrays in any order.

The **Union** of two arrays is a collection of all **distinct elements** present in either of the arrays. If an element appears more than once in one or both arrays, it should be included **only once** in the result.

Note: Elements of **a[]** and **b[]** are not necessarily distinct.
Note that, You can return the Union in any order but the driver code will print the result in **sorted order** only.

Examples:

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1111 / 1111

Attempts : Correct / Total
2 / 3
Accuracy : 66%

Time Taken

Java (21) Start Timer

```java
import java.util.ArrayList;
import java.util.HashSet;

class Solution {
    public static ArrayList<Integer> findUnion(int[] a, int[] b) {
        HashSet<Integer> set = new HashSet<>();

        for (int num : a) {
            set.add(num);
        }
        for (int num : b) {
            set.add(num);
        }

        return new ArrayList<>(set);
    }
}
```

Custom Input Compile & Run Submit

Largest in Array

Difficulty: **Basic** Accuracy: **67.48%** Submissions: **575K+** Points: **1** Average Time: **20m**

Given an array **arr[].** The task is to find the largest element and return it.

Examples:

Input: arr[] = [1, 8, 7, 56, 90]
Output: 90
Explanation: The largest element of the given array is 90.

Input: arr[] = [5, 5, 5, 5]
Output: 5
Explanation: The largest element of the given array is 5.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1115 / 1115

Attempts : Correct / Total
2 / 2
Accuracy : 100%

Time Taken

Java (21) Start Timer

```java
class Solution {
    public int largest(int[] arr) {
        int max = arr[0];

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) {
                max = arr[i];
            }
        }

        return max;
    }
}
```

Custom Input Compile & Run Submit

Rotate Array by One

Difficulty: **Basic** Accuracy: **69.6%** Submissions: **358K+** Points: **1** Average Time: **20m**

Given an array **arr**, rotate the array by one position in clockwise direction.

Examples:

Input: arr[] = [1, 2, 3, 4, 5]
Output: [5, 1, 2, 3, 4]
Explanation: If we rotate arr by one position in clockwise 5 come to the front and remaining those are shifted to the end.

Input: arr[] = [9, 8, 7, 6, 4, 2, 1, 3]
Output: [3, 9, 8, 7, 6, 4, 2, 1]

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1115 / 1115

Attempts : Correct / Total
2 / 2
Accuracy : 100%

Time Taken

Java (21) Start Timer

```java
class Solution {
    public void rotate(int[] arr) {
        int n = arr.length;

        int last = arr[n - 1];
        for (int i = n - 1; i > 0; i--) {
            arr[i] = arr[i - 1];
        }

        arr[0] = last;
    }
}
```

Custom Input Compile & Run Submit

Kadane's Algorithm

Difficulty: **Medium** Accuracy: **36.28%** Submissions: **1.2M** Points: **4** Average Time: **20m**

You are given an integer array **arr[]**. You need to find the **maximum** sum of a subarray (containing at least one element) in the array **arr[]**.

Note : A **subarray** is a continuous part of an array.

Examples:

Input: arr[] = [2, 3, -8, 7, -1, 2, 3]
Output: 11
Explanation: The subarray [7, -1, 2, 3] has the largest sum 11.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1120 / 1120

Attempts : Correct / Total
2 / 2
Accuracy : 100%

Time Taken

Java (21) Start Timer

```java
class Solution {
    int maxSubarraySum(int[] arr) {
        int maxSoFar = arr[0];
        int currSum = arr[0];

        for (int i = 1; i < arr.length; i++) {
            currSum = Math.max(arr[i], currSum + arr[i]);
            maxSoFar = Math.max(maxSoFar, currSum);
        }

        return maxSoFar;
    }
}
```

Custom Input Compile & Run Submit

35. Search Insert Position

Solved

Easy | Topics | Companies

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [1,3,5,6], target = 5
Output: 2
```

Example 2:

```
Input: nums = [1,3,5,6], target = 2
Output: 1
```

Example 3:

```
Input: nums = [1,3,5,6], target = 7
Output: 4
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` contains **distinct** values sorted in **ascending** order.

Code

Java Auto

```java
class Solution {
    public int searchInsert(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target) {
                return mid;
            } else if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
```

Saved Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[1,3,5,6]

target =
5

Output

2

1. Two Sum

Solved

Easy | Topics | Companies | Hint

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
```

Example 2:

```
Input: nums = [3,2,4], target = 6
Output: [1,2]
```

Example 3:

```
Input: nums = [3,3], target = 6
Output: [0,1]
```

Constraints:

- `2 <= nums.length <= 10^4`
- `-10^9 <= nums[i] <= 10^9`

67.8K 1.9K
1769 Online

Code

Java Auto

```java
import java.util.HashMap;
import java.util.Map;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];

            if (map.containsKey(complement)) {
                return new int[] { map.get(complement), i };
            }
```

Saved Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[2,7,11,15]

target =
9

Output

[0,1]

Minimum Jumps

Difficulty: **Medium** Accuracy: **11.91%** Submissions: **1.1M** Points: **4**

You are given an array **arr[]** of non-negative numbers. Each number tells you the **maximum number of steps** you can jump forward from that position.

For example:

- If **arr[i] = 3**, you can jump to index **i + 1**, **i + 2**, or **i + 3** from position **i**.
- If **arr[i] = 0**, you **cannot jump forward** from that position.

Your task is to find the **minimum number of jumps** needed to move from the **first** position in the array to the **last** position.

Note: Return **-1** if you can't reach the end of the array.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1120 / 1120

Attempts : Correct / Total

4 / 4

Accuracy : 100%

Time Taken

Java (21) Start Timer

```java
class Solution {
    public int minJumps(int[] arr) {
        int n = arr.length;

        if (arr[0] == 0) return -1;

        int maxReach = arr[0];
        int steps = arr[0];
        int jumps = 1;

        for (int i = 1; i < n; i++) {
            if (i == n - 1) return jumps;

            maxReach = Math.max(maxReach, i + arr[i]);
            steps--;

            if (steps == 0) {
                jumps++;

                if (i >= maxReach) return -1;

                steps = maxReach - i;
            }
        }

        return -1;
    }
}
```

Custom Input Compile & Run Submit